CENTRO PAULA SOUZA
FACULDADE DE TECNOLOGIA DE FRANCA
DR THOMAZ NOVELINO

TECNOLOGIA EM DESENVOLVIMENTO DE
SOFTWARE MULTIPLATAFORMA
6º SEMESTRE

ANA JÚLIA ALVES MOTA
LAUANE GABRIELA DE ARAÚJO TOLEDO
PEDRO HENRIQUE CINTRA SILVA

RELATÓRIO DE MINERAÇÃO DE DADOS DO DOEFÁCIL

Preparação da base Give Food e agrupamento inicial

Disciplina Mineração de Dados
Projeto Interdisciplinar Grupo 06

FRANCA SP
2026

SUMÁRIO

1 OBJETIVO E ORIGEM DOS DADOS.....3

2 PREPARAÇÃO DOS DADOS.....4

3 RESULTADOS DA PREPARAÇÃO.....5

4 MODELAGEM COM APRENDIZADO NÃO SUPERVISIONADO.....6

5 AVALIAÇÃO DO AGRUPAMENTO.....7

6 LIMITES E CONTINUIDADE DO ESTUDO.....8

7 AMBIENTE E REPRODUÇÃO.....9

REFERÊNCIAS.....10

APÊNDICE A CÓDIGO DE PREPARAÇÃO.....11

APÊNDICE B CÓDIGO DE AGRUPAMENTO.....12

1 OBJETIVO E ORIGEM DOS DADOS

O DoeFácil é um projeto multiplataforma para organizar campanhas, doações e estoque de uma instituição social. Nesta etapa da disciplina de Mineração de Dados, preparamos uma base real de instituições e aplicamos um agrupamento inicial por localização. O tratamento preservou 1.023 registros, e o K-Means formou três grupos.

A análise usa o arquivo givefoodbasededados.json, disponível na raiz do repositório. Sua origem é a API pública Give Food, que reúne informações sobre bancos de alimentos e doações solicitadas. Cada registro representa uma instituição, e não uma doação do aplicativo (GIVE FOOD, s.d.).

A cópia analisada contém 17 campos principais, incluindo identificador, nome, território, situação de funcionamento e coordenadas. Há cadastros da Inglaterra, Escócia, País de Gales e Irlanda do Norte, além de Guernsey, Ilha de Man e Jersey. Estes três últimos territórios são dependências da Coroa.

Tabela 1 Identificação da base e da versão do projeto

Informação	Registro
Fonte original	Give Food API versão 2
Arquivo local	givefoodbasededados.json
Unidade de análise	Cadastro de instituição
Quantidade de registros	1.023
Entrada do arquivo no Git	01/09/2026
Data da coleta original	Não registrada no arquivo
Execução conferida pelo grupo	15/10/2026
Versão consultada	feat/lauane-mineracao-sprint2 21c5c59

Fonte: arquivo JSON, histórico do Git e execução registrada pelo grupo.

O estudo acompanha a definição do problema, a compreensão da fonte, a preparação, a modelagem, a avaliação e a disponibilização dos resultados. O objetivo imediato é observar a distribuição geográfica dos cadastros. A aplicação da mineração ao estoque do DoeFácil depende de dados operacionais coletados ao longo do uso do sistema.

2 PREPARAÇÃO DOS DADOS

O script mining/tratar.py usa a cópia local para que o grupo trabalhe com os mesmos registros. A URL registrada identifica a origem; a execução não baixa uma nova base da internet.

2.1 Leitura e organização

A leitura usa codificação UTF-8. json.loads converte o texto JSON em estruturas do Python, e pandas.json_normalize organiza os registros em uma tabela, transformando campos aninhados em colunas. A variável total guarda a contagem inicial.

2.2 Identificação de duplicações

drop_duplicates remove repetições pelo campo id, conservando a primeira ocorrência de cada identificador. A diferença entre as contagens anterior e posterior informa quantas duplicações foram removidas. Nesta cópia, o resultado foi zero.

2.3 Conversão das coordenadas e ajuste dos nomes

O campo lat_lng é separado na primeira vírgula. to_numeric converte as duas partes em números, e errors="coerce" transforma conteúdos não conversíveis em valores ausentes. No nome, fillna substitui ausências por texto vazio e strip remove espaços nas pontas (PANDAS, s.d.).

2.4 Filtragem e exportação

O filtro mantém closed igual a False, nome preenchido, latitude entre -90 e 90 e longitude entre -180 e 180. Todas as condições precisam ser atendidas. Coordenadas ausentes não passam no filtro. Posições inválidas não são substituídas por estimativas.

O CSV final contém id, name, country, latitude e longitude, sem o índice interno do pandas. Um JSON registra a origem, as contagens e a assinatura SHA-256, que identifica os bytes da cópia utilizada.

3 RESULTADOS DA PREPARAÇÃO

Tabela 2 Contagem dos registros após as verificações

Verificação	Quantidade
Registros lidos do JSON	1.023
Duplicações por id removidas	0
Cadastros fechados ou inválidos removidos	0
Registros exportados para CSV	1.023

Fonte: mining/outputs/tratamento.json e execução de 15/10/2026.

Todos os registros passaram pelas regras usadas. Mesmo sem excluir linhas, o script realizou tratamento: organizou a estrutura, verificou identificadores, converteu as coordenadas, ajustou os nomes e selecionou as colunas. A aprovação nessas regras não comprova que todos os dados da fonte estejam corretos ou atualizados.

Tabela 3 Campos presentes no CSV tratado

Campo	Conteúdo e uso
id	Identificador usado para reconhecer o cadastro
name	Nome da instituição para consulta dos resultados
country	Categoria territorial informada pela fonte
latitude	Coordenada numérica usada pelo agrupamento
longitude	Coordenada numérica usada pelo agrupamento

Fonte: seleção de colunas de mining/tratar.py.

A preparação gera mining/data/instituicoes_tratadas.csv e mining/outputs/tratamento.json. Telefones, e-mails e endereços detalhados não entram no recorte usado neste agrupamento.

A assinatura SHA-256 informada pela execução no Windows foi:

```
7068c9749ed80ffb2691e11b328a5599
ea0aa20918dc4b3de566bdee4df81461
```

As duas linhas acima formam uma única assinatura de 64 caracteres. Mudanças nos bytes, inclusive nos finais de linha, podem alterar o hash sem mudar os registros. Por isso, também conferimos o conteúdo e as contagens de saída.

4 MODELAGEM COM APRENDIZADO NÃO SUPERVISIONADO

O script mining/cluster.py usa o CSV preparado para aplicar K-Means. Esse algoritmo faz parte do aprendizado não supervisionado: as instituições são agrupadas pela semelhança das entradas, sem receber uma coluna com o grupo correto de cada cadastro.

Apenas latitude e longitude entram no modelo. StandardScaler calcula a média e o desvio padrão de cada coluna e padroniza os valores. Assim, o agrupamento considera as duas variáveis em escalas comparáveis. Essa transformação acontece em memória; as coordenadas originais continuam no CSV exportado (SCIKIT-LEARN, s.d.b).

Tabela 4 Configuração definida em cluster.py

Configuração	Valor utilizado
Variáveis de entrada	Latitude e longitude
Transformação	StandardScaler
Algoritmo	K-Means
Número de grupos	3
Semente aleatória	random_state=42
Inicializações	n_init=10

Fonte: mining/cluster.py.

O número três foi definido para uma análise preliminar. random_state=42 fixa a semente da inicialização para favorecer a repetição do resultado. n_init=10 executa dez inicializações e conserva a solução com menor soma das distâncias quadráticas aos centros dos grupos, chamada inércia (SCIKIT-LEARN, s.d.a).

fit_predict ajusta o modelo e retorna os rótulos 0, 1 ou 2. Esses números identificam os grupos e não representam ordem de necessidade. O script exige ao menos quatro instituições e verifica se foram formados três grupos distintos antes de calcular a silhueta.

O resultado é salvo em mining/outputs/instituicoes_clusters.csv, com a coluna cluster acrescentada, em mining/outputs/resumo.json e em mining/outputs/clusters.png. O gráfico apresenta longitude no eixo horizontal e latitude no vertical.

5 AVALIAÇÃO DO AGRUPAMENTO

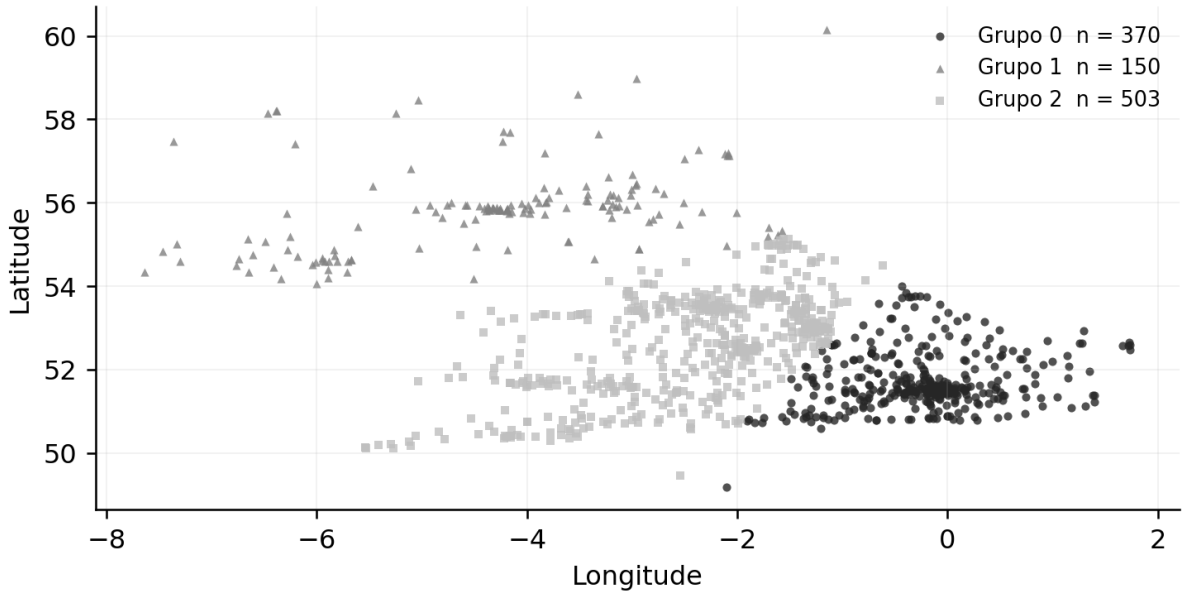
A execução formou três grupos e preservou as 1.023 instituições. O coeficiente de silhueta foi 0,42655731672335884, arredondado para 0,4266. A medida compara a distância dos registros ao próprio grupo com a distância ao grupo vizinho mais próximo (SCIKIT-LEARN, s.d.c).

Tabela 5 Distribuição dos registros por grupo

Grupo	Instituições
0	370
1	150
2	503
Total	1.023

Fonte: mining/outputs/resumo.json.

Figura 1 Distribuição das instituições conforme os grupos obtidos



Fonte: elaboração a partir de mining/outputs/instituicoes_clusters.csv. A figura apresenta as coordenadas originais; o modelo utilizou coordenadas padronizadas.

A silhueta varia de -1 a 1. Valores próximos de 1 indicam maior separação, e valores próximos de zero indicam sobreposição. O resultado de 0,4266 não é uma porcentagem de acerto. Ele descreve a organização desta solução e precisa ser comparado com outras configurações antes de afirmar que três grupos são a melhor escolha.

6 LIMITES E CONTINUIDADE DO ESTUDO

6.1 Alcance dos resultados

O agrupamento explora a distribuição dos cadastros da Give Food. Mais instituições em um grupo não comprovam maior demanda, vulnerabilidade ou número de pessoas atendidas. Os resultados também não descrevem a população de Franca.

As distâncias padronizadas não equivalem a quilômetros ou rotas. Os filtros verificam limites numéricos, sem conferir cada endereço em um mapa. A falta da data original da coleta limita afirmações sobre a situação atual das instituições.

6.2 Continuidade da modelagem

O próximo refinamento será comparar diferentes quantidades de grupos pela silhueta e pela distribuição dos registros. Se a pergunta passar a envolver distância física, também será necessário avaliar uma representação geográfica adequada.

O aprendizado supervisionado ainda precisa ser desenvolvido. Uma possibilidade é classificar se um item ficará abaixo do estoque mínimo em um período futuro. Isso exige histórico real de entradas, saídas, datas, itens, unidades e o resultado observado depois do período. A base cadastral Give Food não oferece esse histórico.

As informações anteriores à previsão devem formar as entradas, e o resultado posterior deve formar o alvo. A avaliação deverá separar períodos de treino e teste e comparar o modelo com uma regra simples. Esta execução não inclui classificação supervisionada.

6.3 Disponibilização e uso no PI

Os scripts, CSVs, resumos e gráfico ficam no repositório para consulta e apresentação. A mineração roda localmente e ainda não está conectada à API ou às telas. O PostgreSQL do aplicativo guarda os dados operacionais. O uso de resultados da mineração no sistema depende de uma pergunta compatível com os dados e de validação posterior.

7 AMBIENTE E REPRODUÇÃO

A execução foi conferida em 15/10/2026, no Windows com PowerShell. O ambiente virtual usa Python 3.15.0. O arquivo mining/requirements-lock.txt registra as 19 bibliotecas instaladas, com suas versões, e foi enviado no commit 21c5c59.

Tabela 6 Versões principais do ambiente utilizado

Componente	Versão
Python	3.15.0
pip	26.2.1
pandas	3.0.6
scikit-learn	1.9.1
matplotlib	3.11.2
NumPy	2.5.4

Fonte: versões informadas na instalação e em pip freeze.

Para preparar outro computador com essa versão do Python, execute na raiz do repositório, em PowerShell:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r mining/requirements-lock.txt
.\.venv\Scripts\python.exe mining/tratar.py
.\.venv\Scripts\python.exe mining/cluster.py
```

Se o ambiente virtual já estiver criado e com as dependências instaladas, os dois últimos comandos executam a análise. O arquivo da base deve continuar na raiz do projeto. Não é necessário iniciar a API ou conectar o PostgreSQL para rodar esses scripts.

Ao terminar, confira 1.023 registros tratados, zero exclusões e três grupos com 370, 150 e 503 instituições. Os resumos são escritos em mining/outputs. O gráfico é salvo em arquivo, sem abrir uma janela, porque o script usa o modo Agg do matplotlib.

O arquivo de versões é um registro do ambiente testado, gerado por pip freeze. Ele ajuda a repetir a instalação, mas não garante funcionamento em qualquer versão de Python ou sistema operacional (PYPA, s.d.).

REFERÊNCIAS

GIVE FOOD. API versão 2. [s.d.]. Disponível em: https://www.givefood.org.uk/api/2/. Acesso em: 15 out. 2026.

GRUPO 06 DSM. DoeFácil. Scripts e resultados da mineração na versão 21c5c59. 2026. Disponível em: https://github.com/FatecFranca/DSM-G06-PI6-2026-2/tree/21c5c59/mining. Acesso em: 15 out. 2026.

PANDAS. pandas.to_numeric. Documentação da versão 3.0.6. [s.d.]. Disponível em: https://pandas.pydata.org/docs/reference/api/pandas.to_numeric.html. Acesso em: 15 out. 2026.

PYPA. pip freeze. Documentação da versão 26.2.1. [s.d.]. Disponível em: https://pip.pypa.io/en/stable/cli/pip_freeze/. Acesso em: 15 out. 2026.

SCIKIT-LEARN. KMeans. Documentação da versão 1.9.1. [s.d.a]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html. Acesso em: 15 out. 2026.

SCIKIT-LEARN. StandardScaler. Documentação da versão 1.9.1. [s.d.b]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Acesso em: 15 out. 2026.

SCIKIT-LEARN. silhouette_score. Documentação da versão 1.9.1. [s.d.c]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html. Acesso em: 15 out. 2026.

APÊNDICE A CÓDIGO DE PREPARAÇÃO

Arquivo mining/tratar.py, utilizado para preparar a cópia local da base Give Food.

```
from pathlib import Path
import hashlib
import json
import pandas as pd

PASTA = Path(__file__).resolve().parent
FONTE = PASTA.parent / "givefoodbasededados.json"
saida = PASTA / "data"
saida.mkdir(exist_ok=True)
df = pd.json_normalize(json.loads(FONTE.read_text(encoding="utf-8")))
total = len(df)
df = df.drop_duplicates(subset="id").copy()
duplicados = total - len(df)
coordenadas = df["lat_lng"].str.split(",", n=1, expand=True)
df["latitude"] = pd.to_numeric(coordenadas[0], errors="coerce")
df["longitude"] = pd.to_numeric(coordenadas[1], errors="coerce")
df["name"] = df["name"].fillna("").str.strip()
validas = (df["closed"].eq(False) & df["name"].ne("")
    & df["latitude"].between(-90, 90)
    & df["longitude"].between(-180, 180))
limpo = df.loc[validas, ["id", "name", "country", "latitude", "longitude"]]
limpo.to_csv(saida / "instituicoes_tratadas.csv", index=False)
relatorio = {
    "fonte": "givefoodbasededados.json (snapshot existente no repositorio)",
    "origem": "https://www.givefood.org.uk/api/2/foodbanks/",
    "sha256": hashlib.sha256(FONTE.read_bytes()).hexdigest(),
    "registros_brutos": total,
    "duplicados_por_id": duplicados,
    "inativos_ou_invalidos": len(df) - len(limpo),
    "registros_tratados": len(limpo),
}
outputs = PASTA / "outputs"
outputs.mkdir(exist_ok=True)
(outputs / "tratamento.json").write_text(
    json.dumps(relatorio, indent=2, ensure_ascii=False), encoding="utf-8")
print(json.dumps(relatorio, indent=2, ensure_ascii=False))
```

Fonte: código do projeto na versão 21c5c59. Ordem de leitura: importações, caminhos, leitura da base, regras de tratamento, exportação e resumo.

APÊNDICE B CÓDIGO DE AGRUPAMENTO

Arquivo mining/cluster.py, executado depois do pré-processamento.

```
from pathlib import Path
import json
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

PASTA = Path(__file__).resolve().parent
df = pd.read_csv(PASTA / "data" / "instituicoes_tratadas.csv")
if len(df) < 4:
    raise ValueError("Precisa de pelo menos quatro instituicoes validas")
X = StandardScaler().fit_transform(df[["latitude", "longitude"]])
modelo = KMeans(n_clusters=3, random_state=42, n_init=10)
df["cluster"] = modelo.fit_predict(X)
grupos = int(df["cluster"].nunique())
if grupos != 3:
    raise ValueError("A base nao gerou tres grupos distintos")
resumo = {
    "tecnica": "K-Means preliminar por localizacao",
    "k": 3, "registros": len(df),
    "silhouette": float(silhouette_score(X, df["cluster"])),
    "por_grupo": {str(k): int(v) for k, v in df["cluster"].value_counts().items()},
}
saida = PASTA / "outputs"
saida.mkdir(exist_ok=True)
df.to_csv(saida / "instituicoes_clusters.csv", index=False)
(saida / "resumo.json").write_text(
    json.dumps(resumo, indent=2, ensure_ascii=False), encoding="utf-8")
plt.scatter(df["longitude"], df["latitude"], c=df["cluster"], s=8, cmap="viridis")
plt.xlabel("Longitude"); plt.ylabel("Latitude")
plt.title("Give Food: grupos preliminares de instituicoes")
plt.tight_layout(); plt.savefig(saida / "clusters.png", dpi=160); plt.close()
print(json.dumps(resumo, indent=2, ensure_ascii=False))
```

Fonte: código do projeto na versão 21c5c59. Ordem de leitura: importações, leitura do CSV, padronização, K-Means, avaliação e arquivos de saída.